

Chapter Two

Streams and File I/O

Introduction

- Java uses the concept of a **stream** to make **I/O operation fast**.
- I/O means Input/Output.
- The **java.io package** contains all the classes required for input and output operations.
- A stream can be defined as a **sequence of data**.
- An I/O Stream is a sequence of data that is **read** from a **source** and **write** to a **destination**.
- All these streams represent an input source and an output destination.



The Java File Class

- Java File class represents the **files** and **directory pathnames** in an abstract manner.
- The File class is useful for **retrieving information** about **files** or **directories** from disk.
- The **File** class contains the **methods** for obtaining the **properties of a file/directory** and for renaming and deleting a file/directory.
- The File class has different methods which can be used to **manipulate the files**.
 - Absolute path:** An absolute file name (or full name) contains a file name with its **complete path** and **drive letter**.
 - Relative path:** A relative file name is in relation to the **current working directory**.

The File Class

java.io.File

```
+File(pathname: String)
+File(parent: String, child: String)
+File(parent: File, child: String)

+exists(): boolean
+canRead(): boolean
+canWrite(): boolean
+isDirectory(): boolean
+isFile(): boolean
+isAbsolute(): boolean
+isHidden(): boolean

+getAbsolutePath(): String

+getCanonicalPath(): String

+getName(): String

+getPath(): String

+getParent(): String

+lastModified(): long
+length(): long
+listFile(): File[]
+delete(): boolean

+renameTo(dest: File): boolean

+mkdir(): boolean
+makedirs(): boolean
```

Creates a **File** object for the specified path name. The path name may be a directory or a file.

Creates a **File** object for the child under the directory parent. The child may be a file name or a subdirectory.

Creates a **File** object for the child under the directory parent. The parent is a **File** object. In the preceding constructor, the parent is a string.

Returns true if the file or the directory represented by the **File** object exists.

Returns true if the file represented by the **File** object exists and can be read.

Returns true if the file represented by the **File** object exists and can be written.

Returns true if the **File** object represents a directory.

Returns true if the **File** object represents a file.

Returns true if the **File** object is created using an absolute path name.

Returns true if the file represented in the **File** object is hidden. The exact definition of *hidden* is system-dependent. On Windows, you can mark a file hidden in the File Properties dialog box. On Unix systems, a file is hidden if its name begins with a period(.) character.

Returns the complete absolute file or directory name represented by the **File** object.

Returns the same as **getAbsolutePath()** except that it removes redundant names, such as "." and "..", from the path name, resolves symbolic links (on Unix), and converts drive letters to standard uppercase (on Windows).

Returns the last name of the complete directory and file name represented by the **File** object. For example, new **File**("c:\\book\\test.dat").**getName()** returns **test.dat**.

Returns the complete directory and file name represented by the **File** object.

For example, new **File**("c:\\book\\test.dat").**getPath()** returns **c:\\book\\test.dat**.

Returns the complete parent directory of the current directory or the file represented by the **File** object. For example, new **File**("c:\\book\\test.dat").**getParent()** returns **c:\\book**.

Returns the time that the file was last modified.

Returns the size of the file, or 0 if it does not exist or if it is a directory.

Returns the files under the directory for a directory **File** object.

Deletes the file or directory represented by this **File** object. The method returns true if the deletion succeeds.

Renames the file or directory represented by this **File** object to the specified name represented in **dest**. The method returns true if the operation succeeds.

Creates a directory represented in this **File** object. Returns true if the the directory is created successfully.

Same as **mkdir()** except that it creates directory along with its parent directories if the parent directories do not exist.

Example: The File Class

This program demonstrates how to create a **File** object and use the methods in the **File** class to obtain its properties. The program creates a **File** object for the file **us.gif**.

```
1 public class TestFileClass {
2     public static void main(String[] args) {
3         java.io.File file = new java.io.File("image/us.gif");           create a File object
4         System.out.println("Does it exist? " + file.exists());          exists()
5         System.out.println("The file has " + file.length() + " bytes");  length()
6         System.out.println("Can it be read? " + file.canRead());         canRead()
7         System.out.println("Can it be written? " + file.canWrite());     canWrite()
8         System.out.println("Is it a directory? " + file.isDirectory());  isDirectory()
9         System.out.println("Is it a file? " + file.isFile());            isFile()
10        System.out.println("Is it absolute? " + file.isAbsolute());       isAbsolute()
11        System.out.println("Is it hidden? " + file.isHidden());           isHidden()
12        System.out.println("Absolute path is " +
13            file.getAbsolutePath());                                       getAbsolutePath()
14        System.out.println("Last modified on " +
15            new java.util.Date(file.lastModified()));                    lastModified()
16    }
17 }
```

The **lastModified()** method returns the date and time when the file was last modified.

Text File and Binary File

- There are **two** types of files: **Text file** or **Binary file**.
- Data stored in a **Text file** are represented in **human-readable** form.
- Data stored in a **binary file** are represented in **binary form**. You cannot read binary files.
- Binary files are designed to be **read by programs**.
- For example, the Java **source programs** are stored in **text files** and can be read by a **text editor**, but the **Java .class files** are stored in **binary files** and are read by the JVM.
- The advantage of **binary files** I/O are **more efficient to process** than **text files** I/O, because binary file I/O does not require **encoding** and **decoding**. But Text file I/O requires **encoding** and **decoding**.
- For example, the decimal integer **199** is stored as the sequence of three characters: '1', '9', '9' in a **text file** and the same integer is stored as a **byte-type** value C7 in a binary file, because decimal 199 equals to hex C7.

Text File Input and Output

- The **Scanner** class uses for **reading text data** from a file and the **PrintWriter** class uses for **writing text data** to a file.
- *Text data is read using the **Scanner** class and written using the **PrintWriter** class.*
- To create an object using **Scanner** class is:

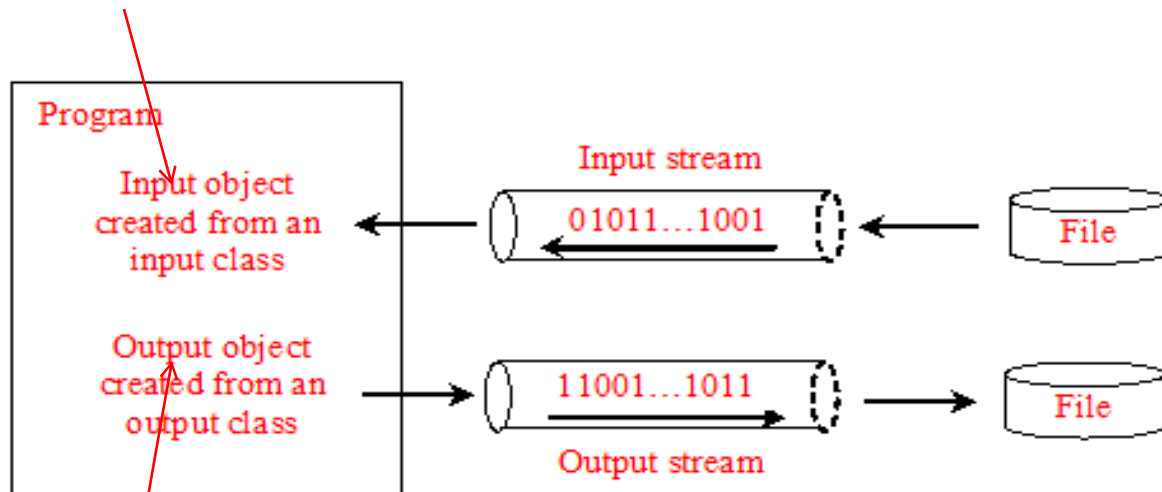
```
Scanner input = new Scanner(new File(filename));
```

- To create an object using **PrintWriter** class is:

```
PrintWriter output = new PrintWriter(filename);
```

File Input and Output

```
Scanner input = new Scanner(new File("temp.txt"));  
System.out.println(input.nextLine());
```



```
PrintWriter output = new PrintWriter("temp.txt");  
output.print("Java 101");  
output.close();
```

Scanner Class

`java.util.Scanner`

```
+Scanner(source: File)
+Scanner(source: String)
+close()
+hasNext(): boolean
+next(): String
+nextLine(): String
+nextByte(): byte
+nextShort(): short
+nextInt(): int
+nextLong(): long
+nextFloat(): float
+nextDouble(): double
+useDelimiter(pattern: String):
  Scanner
```

Creates a `Scanner` that produces values scanned from the specified file.

Creates a `Scanner` that produces values scanned from the specified string.

Closes this scanner.

Returns true if this scanner has more data to be read.

Returns next token as a string from this scanner.

Returns a line ending with the line separator from this scanner.

Returns next token as a `byte` from this scanner.

Returns next token as a `short` from this scanner.

Returns next token as an `int` from this scanner.

Returns next token as a `long` from this scanner.

Returns next token as a `float` from this scanner.

Returns next token as a `double` from this scanner.

Sets this scanner's delimiting pattern and returns this scanner.

PrintWrite Class

java.io.PrintWriter

```
+PrintWriter(file: File)
+PrintWriter(filename: String)
+print(s: String): void
+print(c: char): void
+print(cArray: char[]): void
+print(i: int): void
+print(l: long): void
+print(f: float): void
+print(d: double): void
+print(b: boolean): void
```

Also contains the overloaded
`println` methods.

Also contains the overloaded
`printf` methods.

Creates a `PrintWriter` object for the specified file object.
Creates a `PrintWriter` object for the specified file-name string.
Writes a string to the file.
Writes a character to the file.
Writes an array of characters to the file.
Writes an `int` value to the file.
Writes a `long` value to the file.
Writes a `float` value to the file.
Writes a `double` value to the file.
Writes a `boolean` value to the file.

A `println` method acts like a `print` method; additionally, it prints a line separator. The line-separator string is defined by the system. It is `\r\n` on Windows and `\n` on Unix.

The `printf` method was introduced in §3.16, “Formatting Console Output.”

Example: writing data using PrintWriter

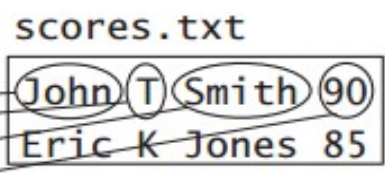
```
1 public class WriteData {
2     public static void main(String[] args) throws IOException {
3         java.io.File file = new java.io.File("scores.txt");
4         if (file.exists()) {
5             System.out.println("File already exists");
6             System.exit(1);
7         }
8
9         // Create a file
10        java.io.PrintWriter output = new java.io.PrintWriter(file);
11
12        // Write formatted output to the file
13        output.print("John T Smith ");
14        output.println(90);
15        output.print("Eric K Jones ");
16        output.println(85);
17
18        // Close the file
19        output.close();
20    }
21 }
```

John T Smith 90
Eric K Jones 85

scores.txt

Example: reading data using Scanner

```
1 import java.util.Scanner;
2
3 public class ReadData {
4     public static void main(String[] args) throws Exception {
5         // Create a File instance
6         java.io.File file = new java.io.File("scores.txt");
7
8         // Create a Scanner for the file
9         Scanner input = new Scanner(file);
10
11        // Read data from a file
12        while (input.hasNext()) {
13            String firstName = input.next();
14            String mi = input.next();
15            String lastName = input.next();
16            int score = input.nextInt();
17            System.out.println(
18                firstName + " " + mi + " " + lastName + " " + score);
19        }
20
21        // Close the file
22        input.close();
23    }
24 }
```



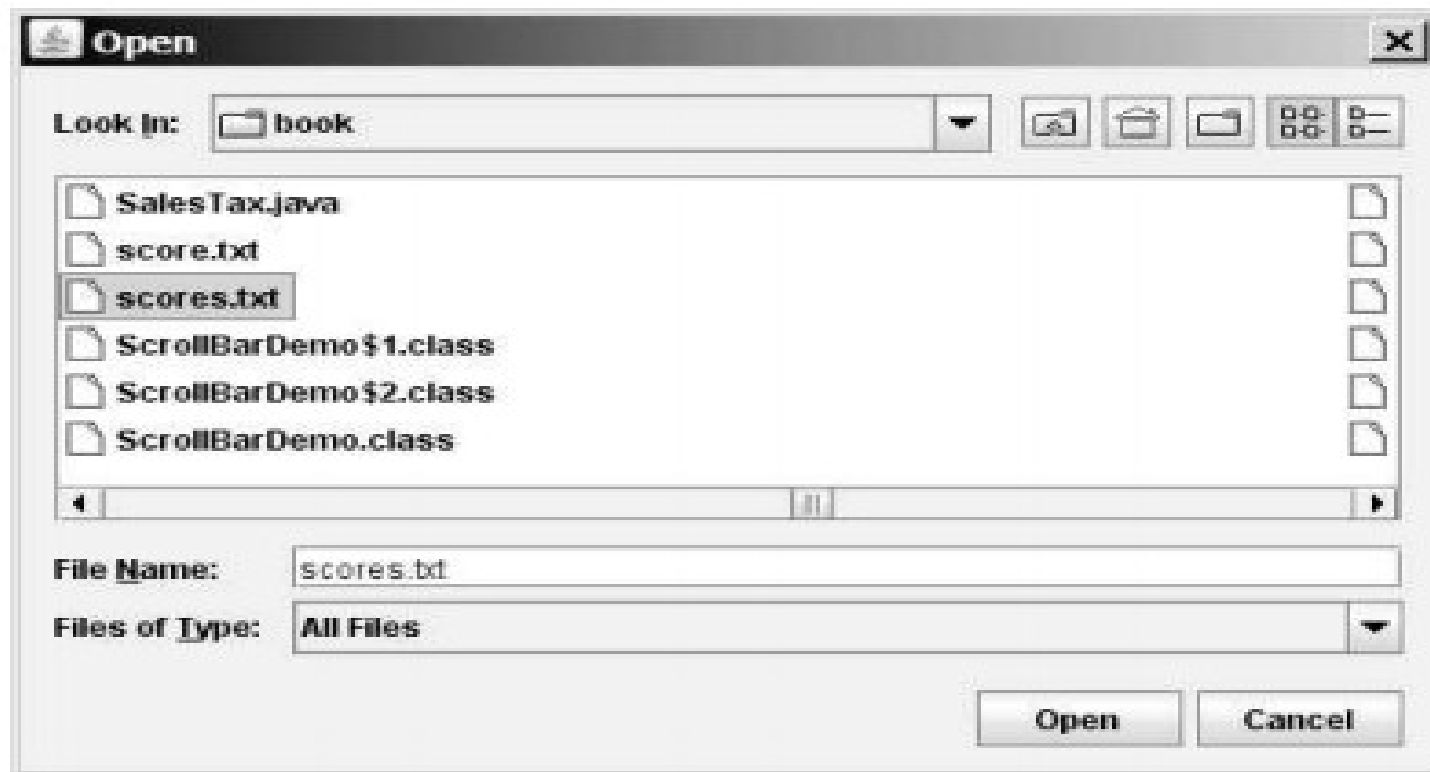
The diagram illustrates how the Scanner methods in the code are used to read data from the file 'scores.txt'. The file contains two lines of text: 'John T Smith 90' and 'Eric K Jones 85'. Arrows point from the following code lines to the corresponding parts of the file content:

- Line 13: `String firstName = input.next();` points to 'John'.
- Line 14: `String mi = input.next();` points to 'T'.
- Line 15: `String lastName = input.next();` points to 'Smith'.
- Line 16: `int score = input.nextInt();` points to '90'.

The second line of the file, 'Eric K Jones 85', is not yet reached by the code shown.

File Dialogs

- The **JFileChooser** class is a GUI component which is used for displaying a file dialog.
- The **JFileChooser** class is found in the java package **javax.swing.JFileChooser**.



Example: File Dialogs

```
1  import java.util.Scanner;
2  import javax.swing.JFileChooser;
3
4  public class ReadFileUsingJFileChooser {
5      public static void main(String[] args) throws Exception {
6          JFileChooser fileChooser = new JFileChooser();
7          if (fileChooser.showOpenDialog(null)
8              == JFileChooser.APPROVE_OPTION) {
9              // Get the selected file
10             java.io.File file = fileChooser.getSelectedFile();
11
12             // Create a Scanner for the file
13             Scanner input = new Scanner(file);
14
15             // Read text from the file
16             while (input.hasNext()) {
17                 System.out.println(input.nextLine());
18             }
19
20             // Close the file
21             input.close();
22         }
23         else {
24             System.out.println("No file selected");
25         }
26     }
27 }
```

Types I/O Streams/Files

- There are three types of I/O streams in Java
 1. Byte I/O Stream
 2. Character I/O Stream
 3. Standard I/O Stream

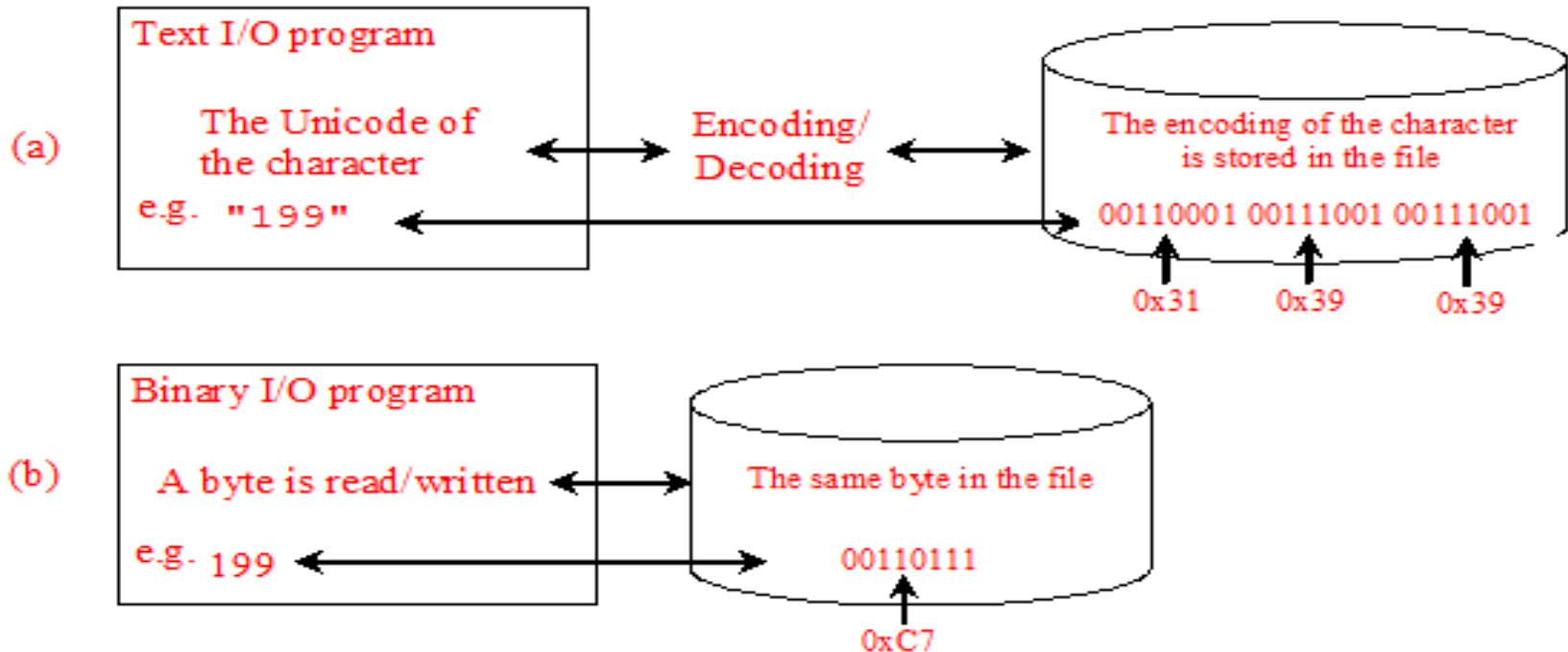
1. Byte I/O Streams

- It is an input and output data in its **binary format**.
- Java byte streams are used to perform input and output of **8-bit** bytes.
- All byte stream classes are descended from two abstract classes: **InputStream** and **OutputStream**.
- The **InputStream** is used to **read data** from a **source** and the **OutputStream** is used for **writing data** to a **destination**.
- There are many classes related to byte streams but the most frequently used classes are, **FileInputStream** and **FileOutputStream**.

1. Byte I/O Streams

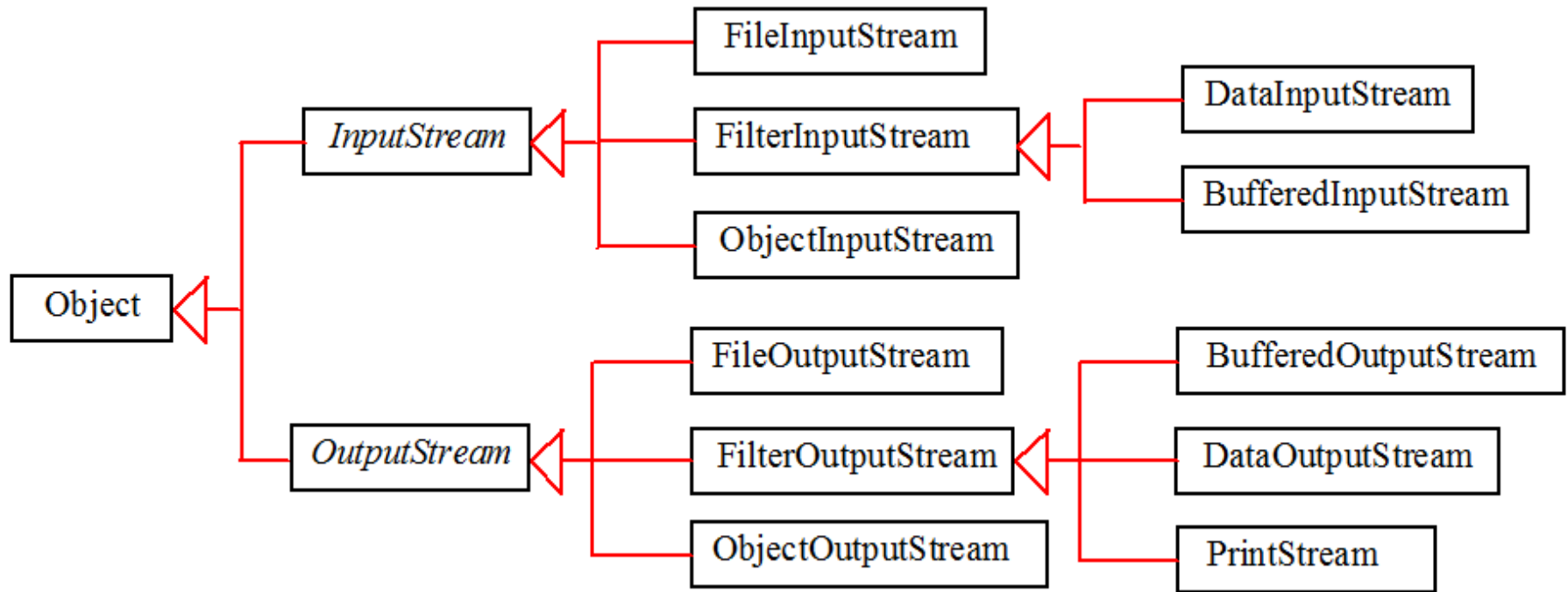
- Computers do not differentiate between **binary files** and **text files**.
- All files are stored in **binary format**, and thus all files are essentially **binary files**. Text I/O requires **encoding** and **decoding**.
- For example, suppose you write the string "199" using text I/O to a file.
- The ASCII or Unicode code for character **1** is **49** (**0x31** in hex) and for character **9** is **57** (**0x39** in hex).
- Thus, to write the characters **199**, three bytes— **0x31**, **0x39**, and **0x39**—are sent to the output, as shown in Figure (a) below.
- Binary I/O does not **require conversions**.
- When you **write a byte** to a file, the **original byte** is copied into the file.
- When you **read a byte** from a file, the **exact byte** in the file is returned.
- For example, a byte-type value **199** is represented as **0xC7** in the memory and appears exactly as **0xC7** in the file, as shown in Figure (b) below.

1. Byte I/O Streams



Binary I/O is **more efficient** than text I/O, because binary I/O **does not** require **encoding** and **decoding**. Binary files are **independent** of the **encoding** scheme on the host machine and thus are portable. Java programs on any machine can read a binary file created by a Java program. This is why Java class files are binary files. Java class files can run on a **JVM on any machine**.

1. Byte I/O Classes



The abstract **InputStream** is the *root class* for *reading byte data* and the abstract **OutputStream** is the *root class* for *writing byte data*.

InputStream

The value returned is a byte as an int type.

java.io.InputStream

+read(): int

Reads the next byte of data from the input stream. The value byte is returned as an int value in the range 0 to 255. If no byte is available because the end of the stream has been reached, the value -1 is returned.

+read(b: byte[]): int

Reads up to b.length bytes into array b from the input stream and returns the actual number of bytes read. Returns -1 at the end of the stream.

+read(b: byte[], off: int, len: int): int

Reads bytes from the input stream and stores into b[off], b[off+1], ..., b[off+len-1]. The actual number of bytes read is returned. Returns -1 at the end of the stream.

+available(): int

Returns the number of bytes that can be read from the input stream.

+close(): void

Closes this input stream and releases any system resources associated with the stream.

+skip(n: long): long

Skips over and discards n bytes of data from this input stream. The actual number of bytes skipped is returned.

+markSupported(): boolean

Tests if this input stream supports the mark and reset methods.

+mark(readlimit: int): void

Marks the current position in this input stream.

+reset(): void

Repositions this stream to the position at the time the mark method was last called on this input stream.

OutputStream

java.io.OutputStream

+write(int b): void

+write(b: byte[]): void

*+write(b: byte[], off: int,
len: int): void*

+close(): void

+flush(): void

Writes the specified byte to this output stream. The parameter `b` is an `int` value. (byte)`b` is written to the output stream.

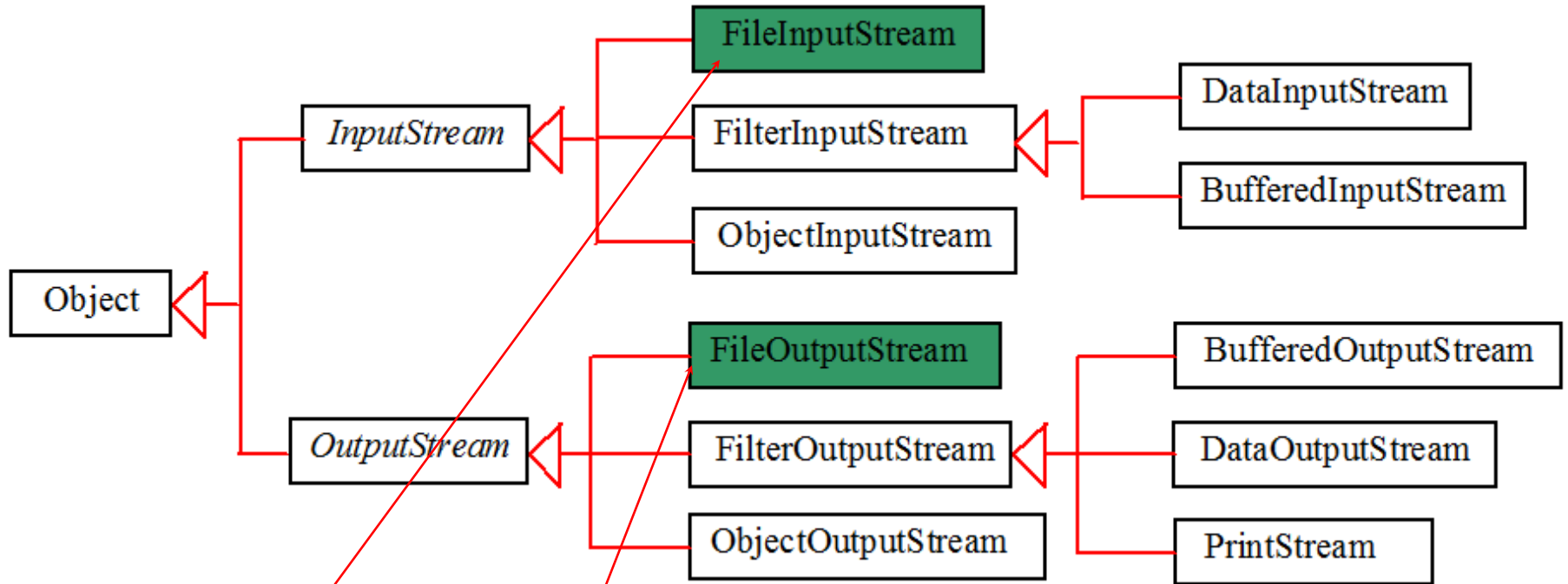
Writes all the bytes in array `b` to the output stream.

Writes `b[off]`, `b[off+1]`, . . . , `b[off+len-1]` into the output stream.

Closes this output stream and releases any system resources occupied by it.

Flushes this output stream and forces any buffered output bytes to be written out.

FileInputStream/FileOutputStream



- `FileInputStream/FileOutputStream` is for **reading/writing bytes** from/to files.
- `FileInputStream/FileOutputStream` associates a **binary input/output stream** with an external file.
- All the methods in `FileInputStream/FileOutputStream` are inherited from its superclasses.

FileInputStream

- To construct a `FileInputStream`, use the following constructors:

```
public FileInputStream(String filename)  
public FileInputStream(File file)
```

- A `java.io.FileNotFoundException` would occur if you attempt to create a `FileInputStream` with a **non-existent file**.

FileOutputStream

- To construct a FileOutputStream, use the following constructors:

```
public FileOutputStream(String filename)
```

```
public FileOutputStream(File file)
```

```
public FileOutputStream(String filename, boolean append)
```

```
public FileOutputStream(File file, boolean append)
```

- If the file does not exist, a new file would be created.
- If the file already exists, the first two constructors would delete the current contents in the file.
- To **retain the current content** and append new data into the file, use the **last two** constructors by passing **true** to the append parameter.

Example: FileInputStream/FileOutputStream

```
import java.io.*;

public class TestFileStream {
    public static void main(String[] args) throws IOException {
        // Create an output stream to the file
        FileOutputStream output = new FileOutputStream("temp.dat");

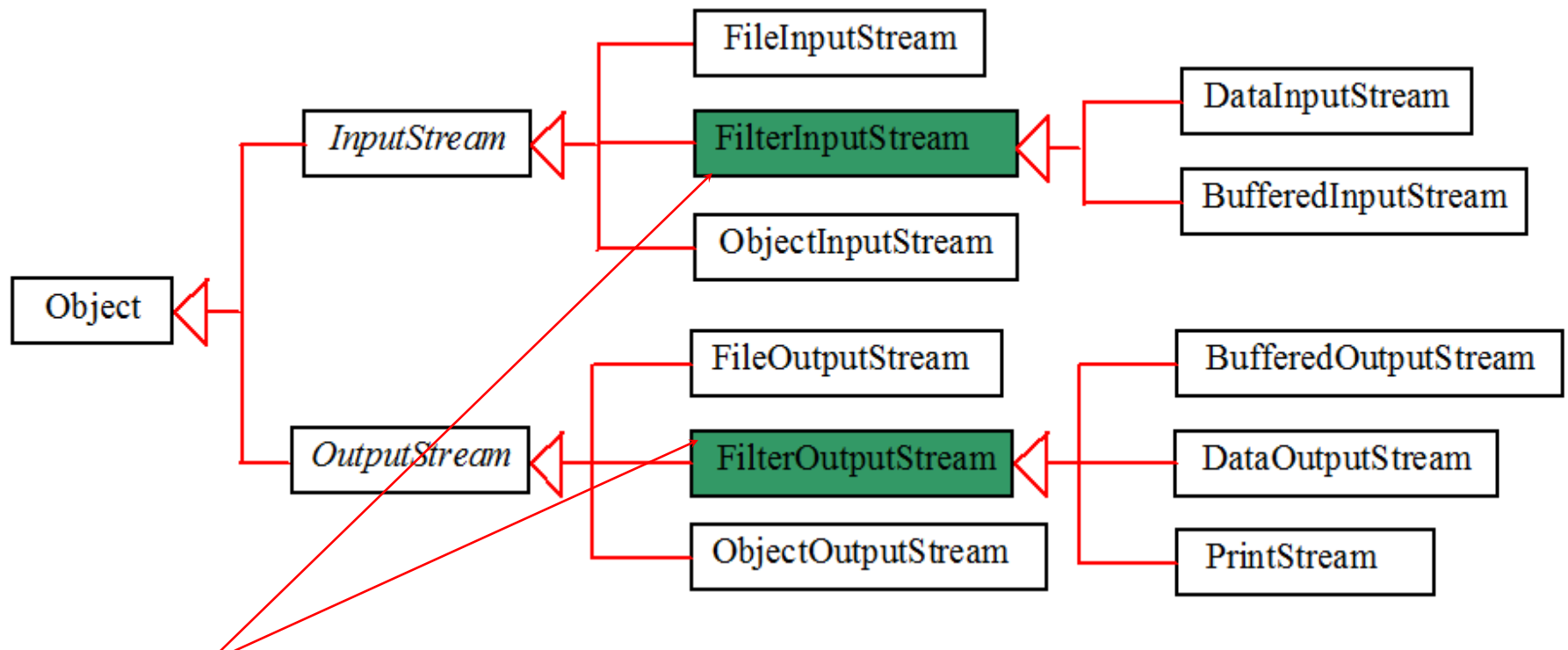
        // Output values to the file
        for (int i = 1; i <= 10; i++)
            output.write(i);
        // Close the output stream
        output.close();

        // Create an input stream for the file
        FileInputStream input = new FileInputStream("temp.dat");
        // Read values from the file
        int value;
        while ((value = input.read()) != -1)
            System.out.print(value + " ");

        // Close the output stream
        input.close();
    }
}
```

This program uses binary I/O to write ten byte values from 1 to 10 to a file named **temp.dat** and reads them back from the file.

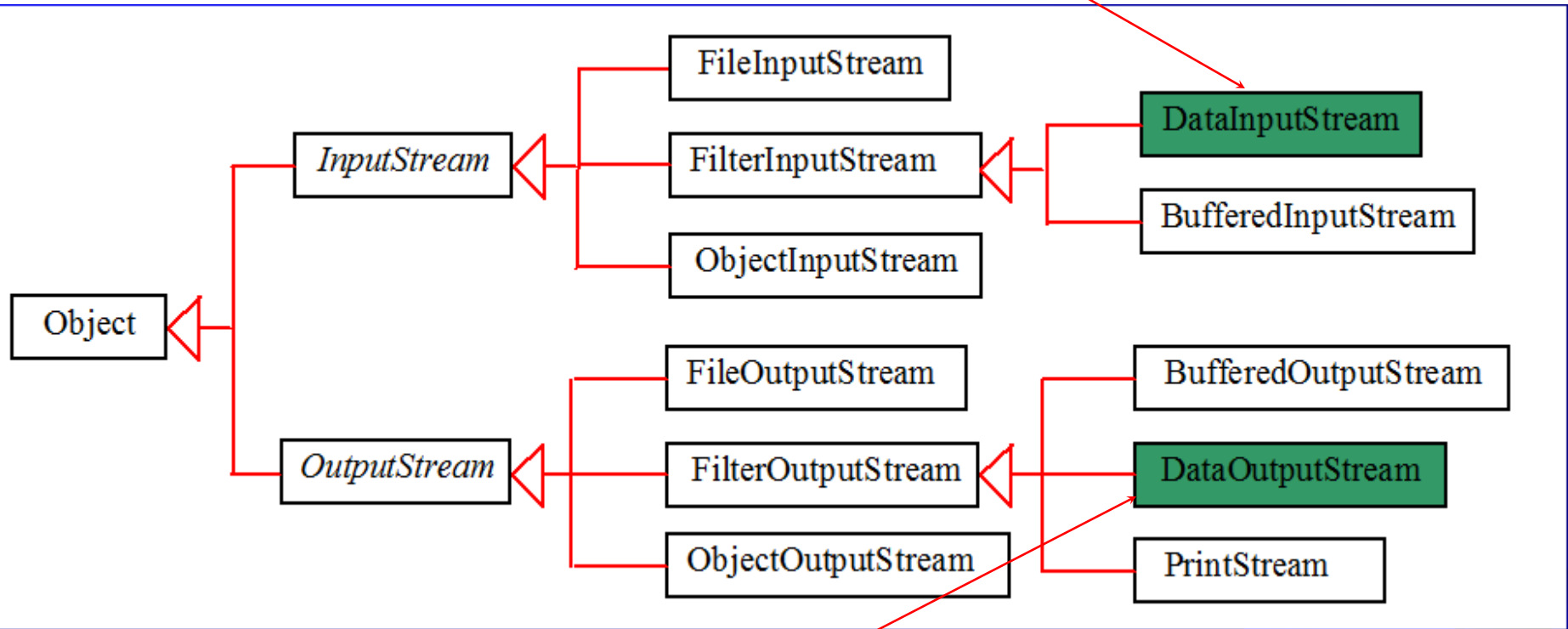
FilterInputStream/FilterOutputStream



Filter streams are streams that **filter bytes** for some purpose. The basic byte input stream provides a **read** method that can only be used for reading bytes. If you want to read **integers**, **doubles**, or **strings**, you need a filter class to wrap the **byte input** stream. Using a filter class enables you to read **integers**, **doubles**, and **strings** instead of **bytes** and **characters**. FilterInputStream and FilterOutputStream are the base classes for filtering data. When you need to process **primitive numeric** types, use DataInputStream and DataOutputStream to filter bytes.

DataInputStream/DataOutputStream

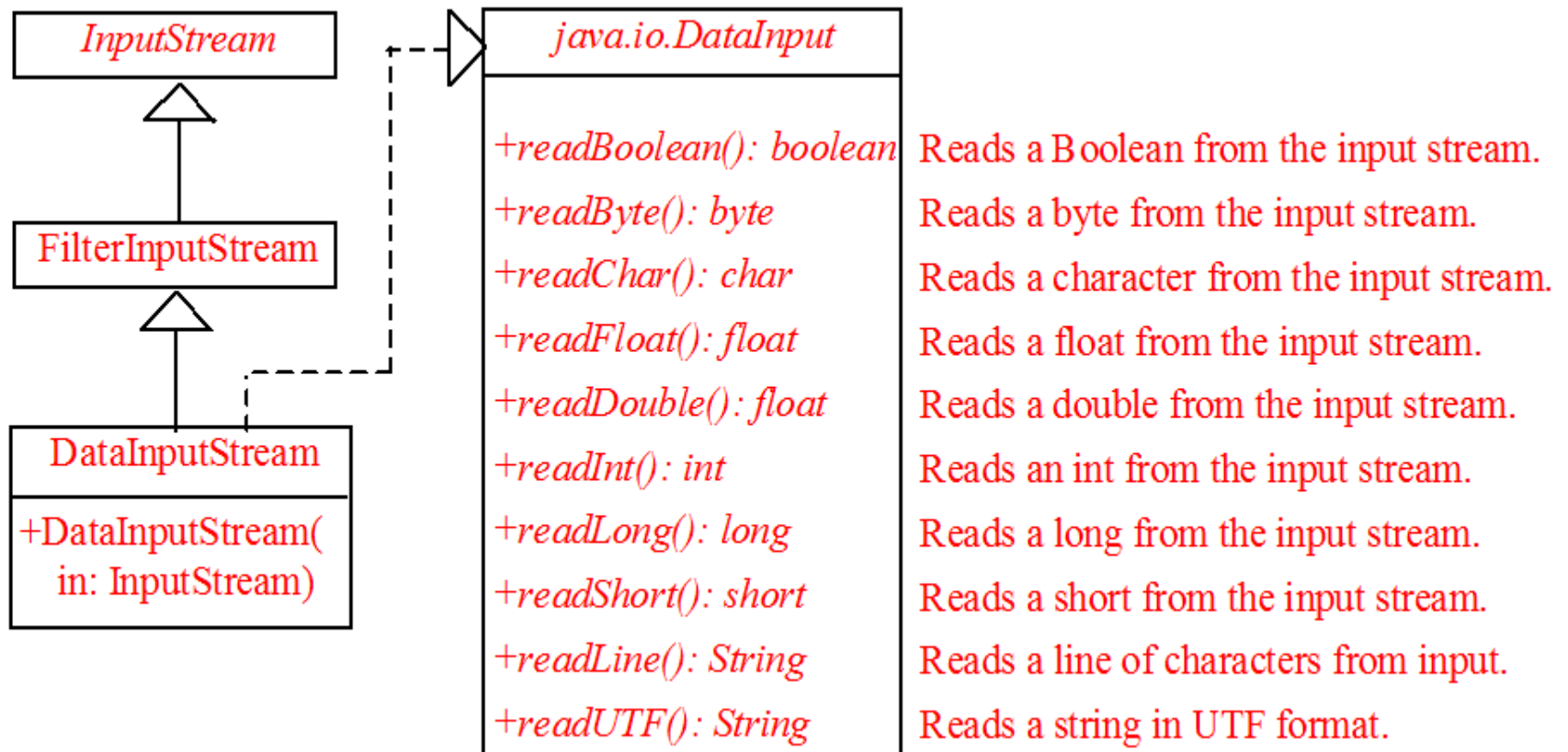
DataInputStream reads bytes from the stream and converts them into appropriate **primitive type** values or strings.



DataOutputStream converts **primitive type** values or strings into **bytes** and output the bytes to the stream.

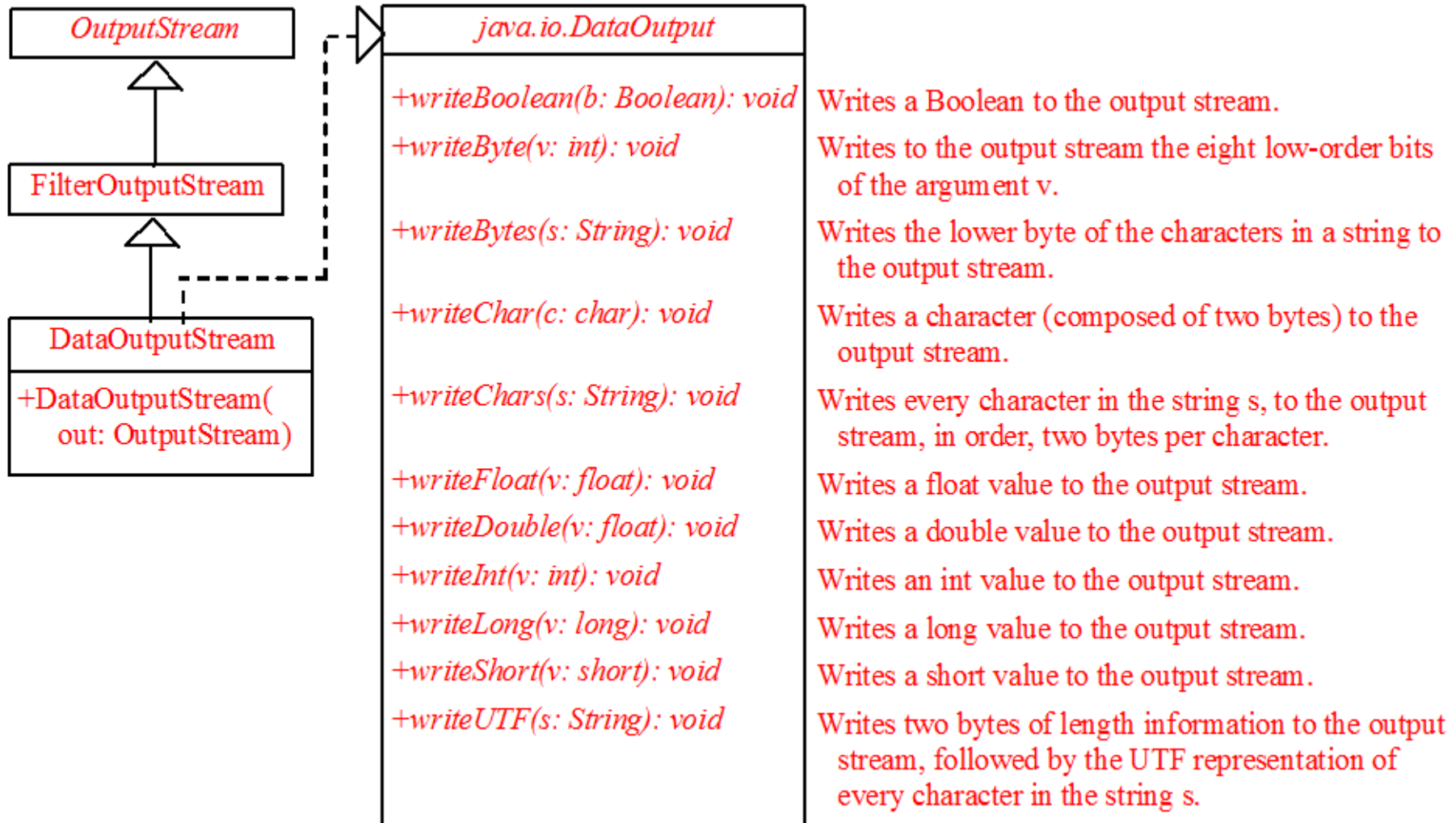
DataInputStream

DataInputStream extends FilterInputStream and implements the DataInput interface.



DataOutputStream

DataOutputStream extends FilterOutputStream and implements the DataOutput interface.



Characters and Strings in Byte I/O

- A **Unicode** consists of **two** bytes. The **writeChar(char c)** method writes the **Unicode of character c** to the output.
- The **writeChars(String s)** method writes the Unicode for each character in the string s to the output.
- The **writeBytes(String s)** method writes the **lower byte** of the **Unicode** for each character in the string s to the output.
- The high byte of the Unicode is discarded.
- The **writeBytes** method is suitable for strings that consist of **ASCII** characters, since an ASCII code is stored only in the **lower byte** of a **Unicode**.
- If a string consists of non-ASCII characters, you have to use the **writeChars** method to write the string.

Characters and Strings in Byte I/O

Why UTF-8? What is UTF-8?

- **UTF-8** is a coding scheme that allows systems to operate with both **ASCII** and **Unicode** efficiently.
- Most operating systems use ASCII. Java uses Unicode. The **ASCII character set** is a **subset** of the **Unicode character set**.
- Since most applications need only the ASCII character set, it is a waste to represent an **8-bit ASCII** character as a **16-bit Unicode** character.
- The UTF-8 is an alternative scheme that stores a character using 1, 2, or 3 bytes.
- ASCII values (less than 0x7F) are coded in one byte. Unicode values less than 0x7FF are coded in two bytes. Other Unicode values are coded in three bytes.

DataInputStream/DataOutputStream

- Data streams are used as wrappers on existing input and output streams to filter data in the original stream. They are created using the following constructors:

```
public DataInputStream(InputStream instream)
```

```
public DataOutputStream(OutputStream outstream)
```

- The statements given below create data streams. The first statement creates an input stream for file **in.dat**; the second statement creates an output stream for file **out.dat**.

```
DataInputStream    infile    =    new    DataInputStream(new  
FileInputStream("in.dat"));
```

```
DataOutputStream outfile = new DataOutputStream(new  
FileOutputStream("out.dat"));
```

Example: DataInputStream/DataOutputStream

```
import java.io.*;

public class TestDataStream {
    public static void main(String[] args) throws IOException {
        // Create an output stream for file temp.dat
        DataOutputStream output = new DataOutputStream(new
        FileOutputStream("temp.dat"));
        // Write student test scores to the file
        output.writeUTF("John");
        output.writeDouble(85.5);
        output.writeUTF("Susan");
        output.writeDouble(185.5);
        output.writeUTF("Kim");
        output.writeDouble(105.25);
        // Close output stream
        output.close();
        // Create an input stream for file temp.dat
        DataInputStream input = new DataInputStream(new
        FileInputStream("temp.dat"));

        // Read student test scores from the file
        System.out.println(input.readUTF() + " " + input.readDouble());
        System.out.println(input.readUTF() + " " + input.readDouble());
        System.out.println(input.readUTF() + " " + input.readDouble());
    }
}
```

This program writes student names and scores to a file named **temp.dat** and reads the data back from the file.

Order and Format

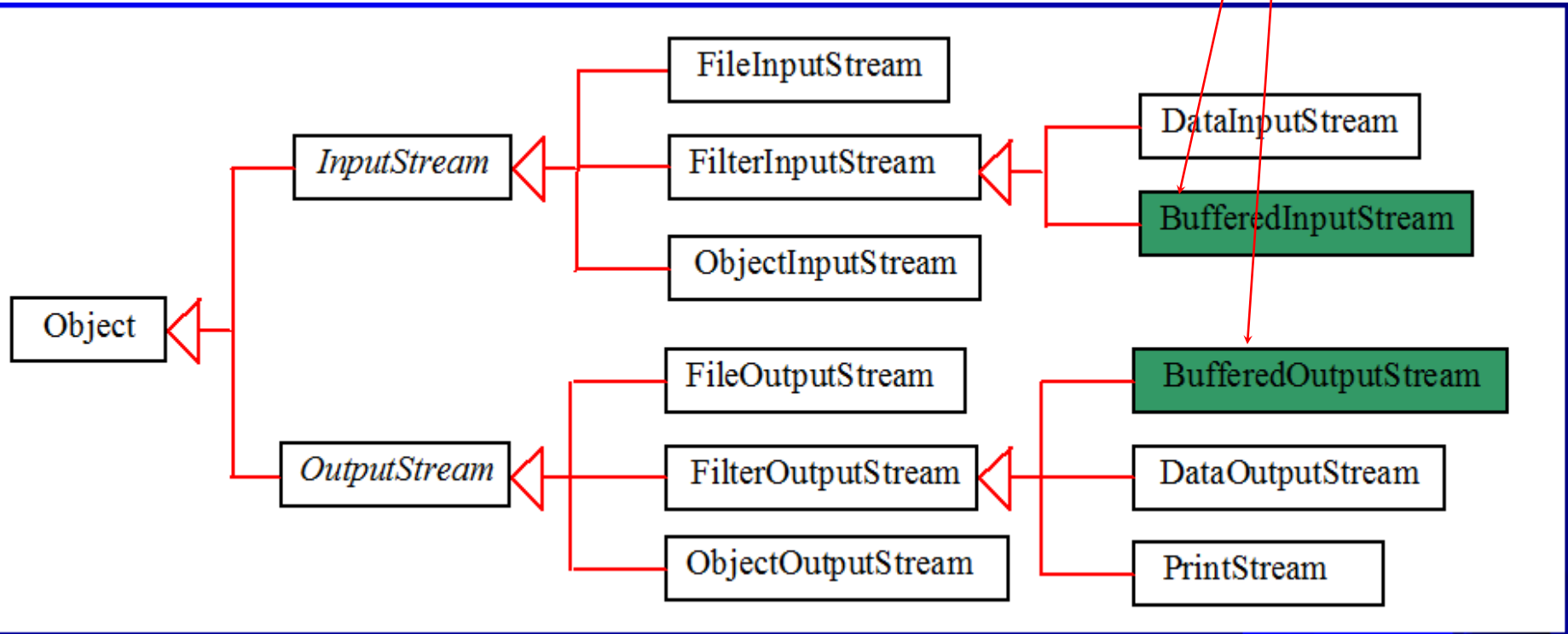
CAUTION: You have to read the data in the same order and same format in which they are stored. For example, since names are written in UTF-8 using writeUTF, you must read names using readUTF.

Checking End of File

TIP: If you keep reading data at the end of a stream, an EOFException would occur. So how do you check the end of a file? You can use input.available() to check it. input.available() == 0 indicates that it is the end of a file.

BufferedInputStream/BufferedOutputStream

Using buffers to speed up I/O

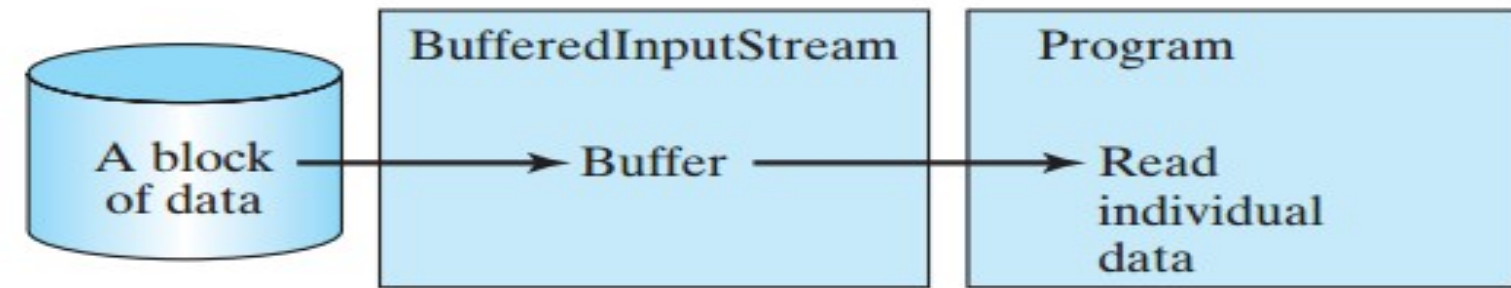


BufferedInputStream/BufferedOutputStream does not contain new methods. All the methods BufferedInputStream/BufferedOutputStream are inherited from the InputStream/OutputStream classes. **BufferedInputStream/BufferedOutputStream** manages a buffer behind the scene and automatically reads/writes data from/to disk on demand.

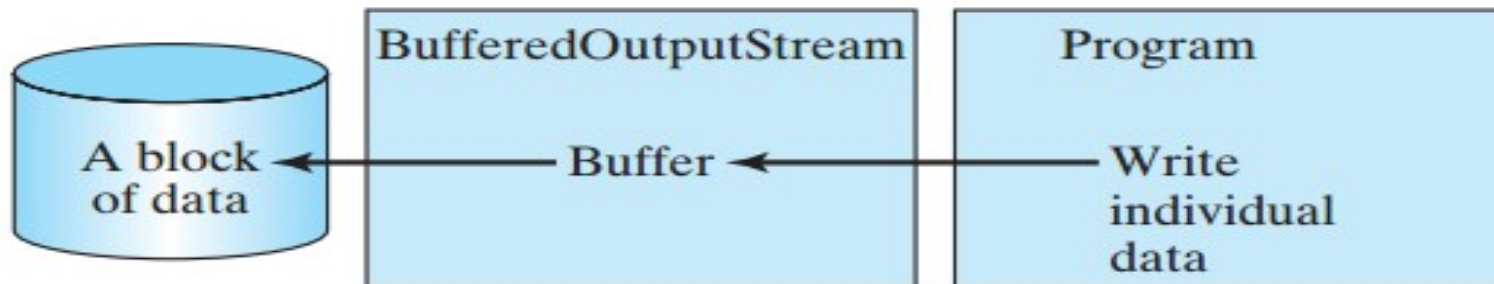
BufferedInputStream/BufferedOutputStream

- **BufferedInputStream/BufferedOutputStream** can be used to **speed up input and output** by reducing the number of disk reads and writes.
- Using **BufferedInputStream**, the whole block of data on the disk is read into the **buffer in the memory once**.
- The individual data are then delivered to your program from the buffer, as shown in Figure (a) below.
- Using **BufferedOutputStream**, the individual data are first written to the **buffer in the memory**.
- When the buffer is full, all data in the buffer is written to the disk once, as shown in Figure (b) below.

BufferedInputStream/BufferedOutputStream



(a)



(b)

Constructing BufferedInputStream/BufferedOutputStream

// Create a BufferedInputStream

```
public BufferedInputStream(InputStream in)
```

```
public BufferedInputStream(InputStream in, int bufferSize)
```

// Create a BufferedOutputStream

```
public BufferedOutputStream(OutputStream out)
```

```
public BufferedOutputStream(OutputStreamr out, int bufferSize)
```

If no buffer size is specified, the default size is **512** bytes.

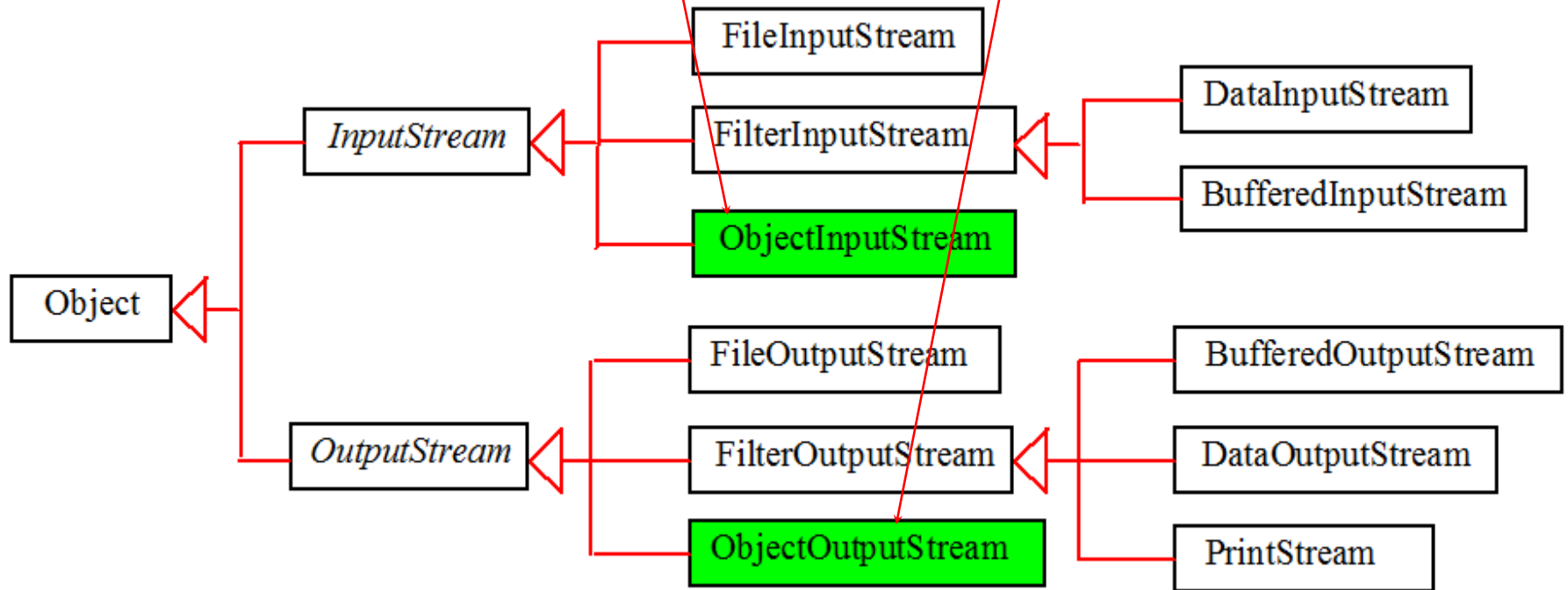
BufferedInputStream/BufferedOutputStream

```
DataInputStream input = new DataInputStream(new  
    BufferedInputStream(new FileInputStream("temp.dat")));
```

```
DataOutputStream output = new DataOutputStream(new  
    BufferedOutputStream (new FileOutputStream("temp.dat")));
```

ObjectInputStream/ObjectOutputStream

DataInputStream/DataOutputStream enables you to perform I/O for **primitive type values** and **strings**. ObjectInputStream/ObjectOutputStream enables you to perform I/O for objects in addition for primitive type values and strings.



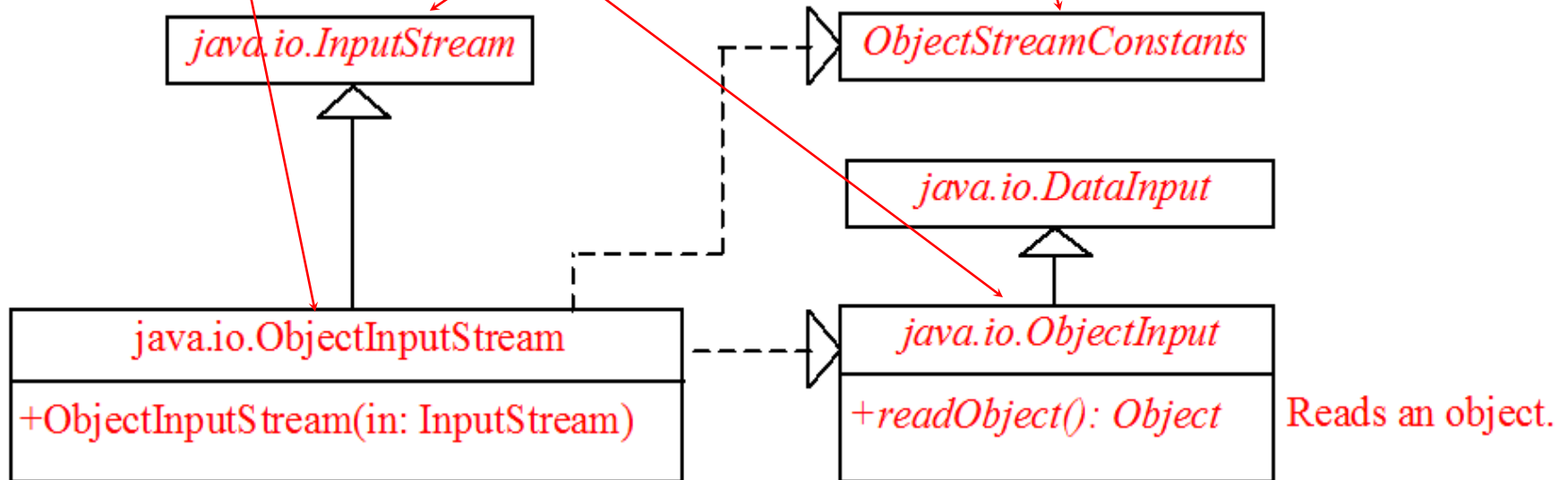
ObjectInputStream/ObjectOutputStream

- **ObjectInputStream/ObjectOutputStream** *classes can be used to read/write serializable objects.*
- **DataInputStream/DataOutputStream** enables you to perform I/O for **primitive type** values and **strings**.
- **ObjectInputStream/ObjectOutputStream** enables you to perform I/O for **objects** in addition to primitive type values and strings.
- Since **ObjectInputStream/ObjectOutputStream** contains all the functions of **DataInputStream/DataOutputStream**, you can replace **DataInputStream/DataOutputStream** completely with **ObjectInputStream/ObjectOutputStream**.

ObjectInputStream

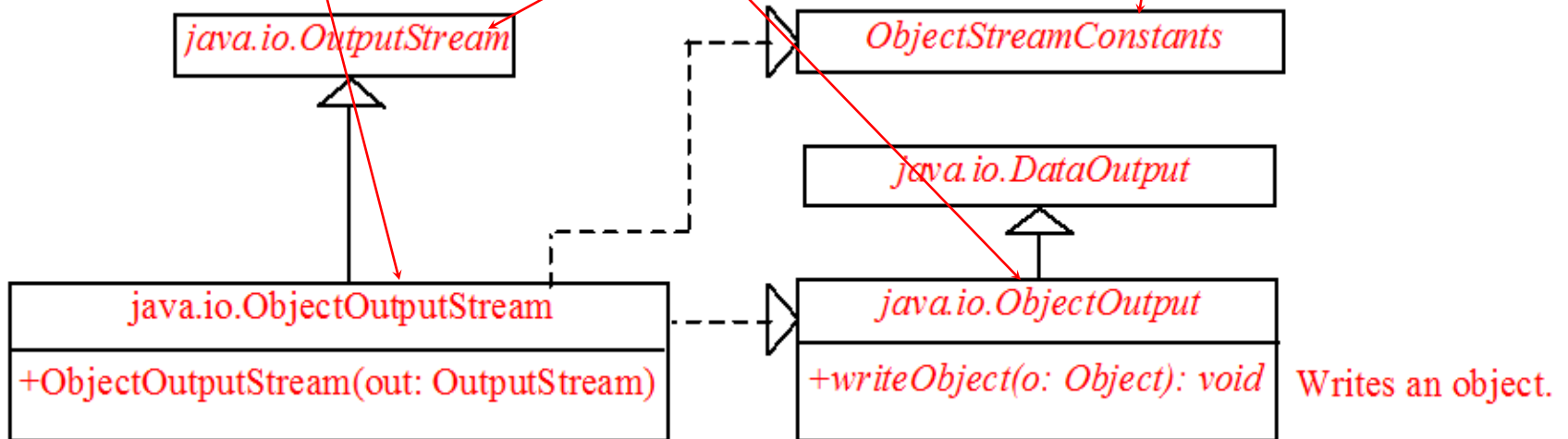
ObjectInputStream extends InputStream and implements ObjectInput and ObjectStreamConstants.

ObjectStreamConstants contains the constants to support **ObjectInputStream/ObjectOutputStream**.



ObjectOutputStream

ObjectOutputStream extends OutputStream and implements ObjectOutputStreamConstants.



Using Object Streams

You may wrap an `ObjectInputStream/ObjectOutputStream` on any `InputStream/OutputStream` using the following constructors:

// Create an ObjectInputStream

```
public ObjectInputStream(InputStream in)
```

// Create an ObjectOutputStream

```
public ObjectOutputStream(OutputStream out)
```

Serialization and Deserialization in Java

- **Serialization** is a process of converting an **object** into a **sequence of bytes** which can be persisted to a disk or database or can be sent through streams.
- **Deserialization** is the process of reconstructing the object from the serialized state.
- It is the reverse process of creating an **object** from **sequence of bytes**.
- An **ObjectInputStream** class deserializes objects and primitive data written using an **ObjectOutputStream**.
- The **ObjectOutputStream** class is used to write **primitive data types** and **Java objects** to sequence of **bytes**.
- Only objects that support the `java.io.Serializable` interface can be written to streams.

Example: Serializable program

```
import java.io.Serializable;
import java.io.*;
public class Student implements Serializable{
    int id;
    String name;
    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
}
class Persist{
    public static void main(String args[])throws Exception{
        Student s1 =new Student(211,"ravi");
        FileOutputStream fout=new FileOutputStream("file.txt");
        ObjectOutputStream out=new ObjectOutputStream(fout);
        out.writeObject(s1);
        out.flush();
        System.out.println("success");
    }
}
```

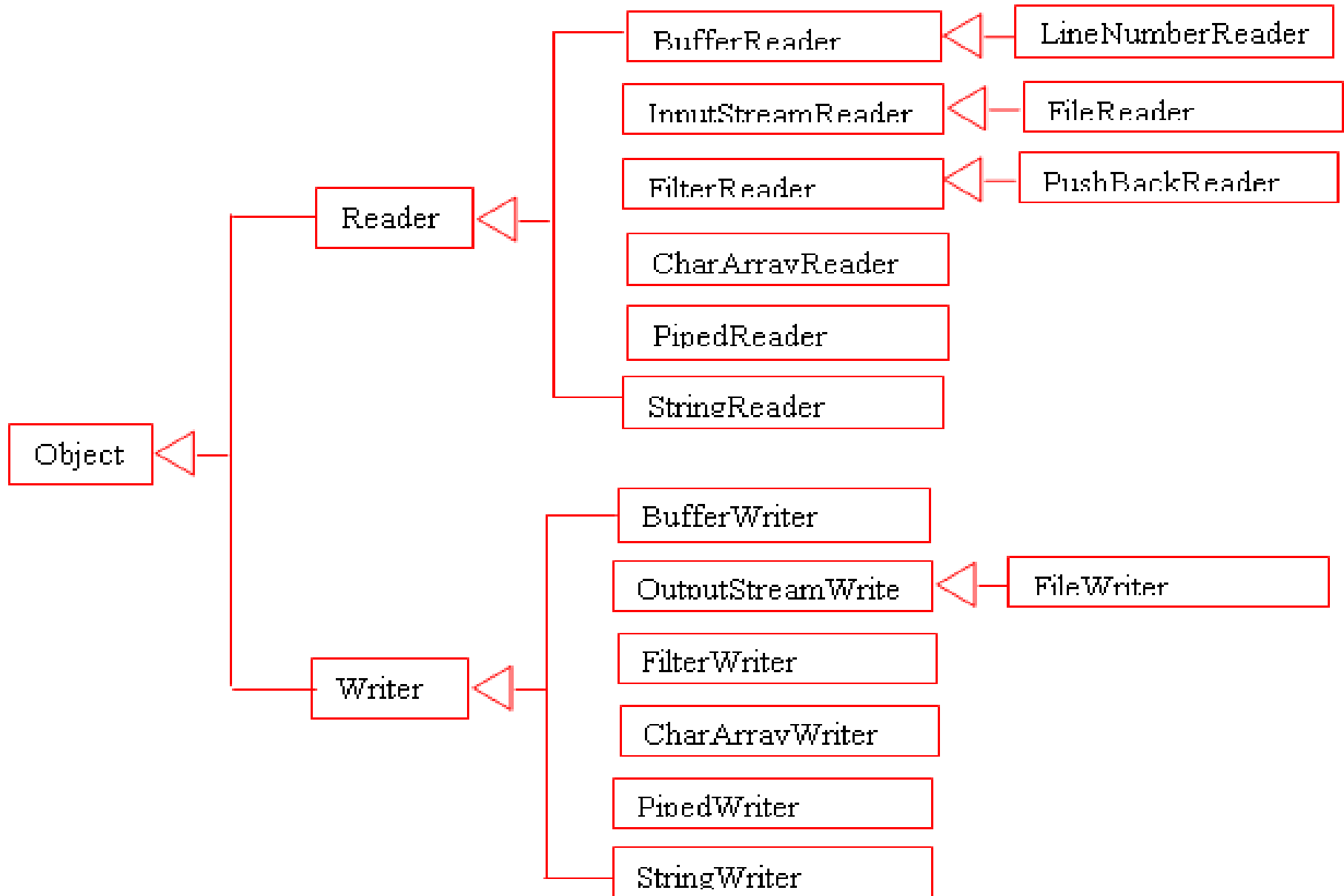
Example: Deserializable program

```
import java.io.*;
public class Depersist{
public static void main(String args[])throws Exception{
    FileInputStream input = new
    FileInputStream("file.txt");
    ObjectInputStream in=new ObjectInputStream(input);
    Student s=(Student)in.readObject();
    System.out.println(s.id+" "+s.name);
    in.close();
}
}
```

2. Character I/O Streams

- Character Stream is an input and output data as a **sequence of characters**.
- Java **Byte** streams are used to perform input and output of **8-bit bytes**, whereas Java **Character** streams are used to perform input and output for **16-bit Unicode**.
- All character stream classes are also descended from two abstract classes **Read** and **Writer**.
- The most frequently used character stream classes are: **FileReader** and **FileWriter**.

2. Character I/O Streams



3. Standard I/O Streams

- All the programming languages provide support for **standard I/O** where the user's program can take input from a **keyboard** and then produce an output on the **computer screen**.
- Java provides the following three standard streams:
 - **Standard Input** - This is used to feed the data to user's program and usually a keyboard is used as standard input stream and represented as **System.in**.
 - **Standard Output** - This is used to output the data produced by the user's program and usually a computer screen is used for standard output stream and represented as **System.out**.
 - **Standard Error** - This is used to output the error data produced by the user's program and usually a computer screen is used for standard error stream and represented as **System.err**.

The End!!